SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Constraint Based Problem Solving – Level 1

COURSE NAME: R PROGRAMMING **COURSE CODE:** ITA04

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively. (Bloom Level – BL2–BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:** 5 × 20 = 100

Question 1: Vectors – Weekly Sales Analysis

A shop records the number of units sold on each day of a week: Monday 12, Tuesday 18, Wednesday 9, Thursday 25, Friday 30, Saturday 14, Sunday 22.

Constraints

• Store the data in a single named vector using c().
• Do not use any loop (for, while, repeat).
• Use logical indexing, not manual selection of positions.
• Round the average to 2 decimal places.

Tasks

(a) Create the named vector of sales.
(b) Calculate the total and the average sales for the week.
(c) Display only the days whose sales are above the weekly average.
(d) Find the name of the day with the highest sales.

Answer:

```
sales <- c(Monday=12, Tuesday=18, Wednesday=9, Thursday=25,
             Friday=30, Saturday=14, Sunday=22)
print(sales)
total <- sum(sales)
avg <- round(mean(sales), 2)
print(total)
print(avg)
sales[sales > avg]
names(sales)[which.max(sales)]
```

Output:

```
   Monday   Tuesday Wednesday  Thursday    Friday  Saturday    Sunday
       12        18         9        25        30        14        22
[1] 130
[1] 18.57
 Thursday    Friday    Sunday
       25        30        22
[1] "Friday"
```

Question 2: Lists – Student Record

A teacher wants to keep one record for a student named Asha, who scored 78, 85 and 92 in three tests and has passed the course.

Constraints

• Use a single list to hold the name (character), marks (numeric vector) and passed (logical).
• Access elements only with $ or [[ ]].
• Add the new element by assignment; do not rebuild the list.
• Do not use a data frame.

Tasks

(a) Create the list named student.
(b) Calculate the average of the marks using the list element.
(c) Add a new element grade: "A" if the average is 85 or above, otherwise "B".
(d) Display the number of elements in the list and their names.

Answer:

```
student <- list(name="Asha", marks=c(78, 85, 92), passed=TRUE)
avg <- mean(student$marks)
print(avg)
student$grade <- if (avg >= 85) "A" else "B"
print(student$grade)
length(student)
names(student)
```

Output:

```
[1] 85
[1] "A"
[1] 4
```

Question 3: Matrices – Operations on Two Matrices

Matrix A holds the numbers 1 to 9 arranged row by row in a 3 × 3 grid. Matrix B is a 3 × 3 matrix with the row-wise values 1, 0, 2, 0, 1, 0, 3, 0, 1.

Constraints

• Create both matrices with matrix() and the byrow = TRUE argument.
• Do not use loops.
• Use * for element-wise multiplication and %*% for matrix multiplication.
• Use base R functions only.

Tasks

(a) Create matrices A and B.
(b) Compute the element-wise product of A and B.
(c) Compute the matrix product of A and B.
(d) Find the transpose of A and the row sums of A.

Answer:

```
A <- matrix(1:9, nrow=3, byrow=TRUE)
B <- matrix(c(1,0,2,0,1,0,3,0,1), nrow=3, byrow=TRUE)
print(A)
print(B)
A*B
A %*% B
t(A)
rowSums(A)
```

Output:

```
A:
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
B:
     [,1] [,2] [,3]
[1,]    1    0    2
[2,]    0    1    0
[3,]    3    0    1

A*B:
     [,1] [,2] [,3]
[1,]    1    0    6
[2,]    0    5    0
[3,]   21    0    9

A %*% B:
     [,1] [,2] [,3]
[1,]   10    2    5
[2,]   22    5   14
[3,]   34    8   23

t(A):
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9

rowSums(A):
[1]  6 15 24
```

Question 4: Arrays – Three-Dimensional Data

A warehouse stores 12 item counts in a 3-dimensional structure with 2 rows, 3 columns and 2 layers. The counts are the numbers 1 to 12.

Constraints

• Create the structure with array() using the dim argument.
• Do not use loops.
• Extract values with comma-separated index notation, for example arr[row, col, layer].
• Use apply() to calculate the layer totals.

Tasks

(a) Create the array named arr with dim = c(2, 3, 2).
(b) Display the entire second layer.
(c) Extract the element in row 2, column 3 of layer 1.
(d) Calculate the sum of each layer.

Answer:

```
arr <- array(1:12, dim=c(2,3,2))
print(arr)
arr[, , 2]
arr[2, 3, 1]
apply(arr, 3, sum)
```

Output:

```
, , 1
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

, , 2
     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12

arr[, , 2]:
     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12

arr[2, 3, 1]:
[1] 6

apply(arr, 3, sum):
[1] 21 57
```

Question 5: Class Types and Coercion – Price List

Prices from a text file were read into R as the character vector "120", "85.5", "200". The shop needs the total bill after a 10% discount.

Constraints

• Do not retype the prices as numbers; convert the existing vector.
• Use as.numeric() for conversion.
• Check the type with class() before and after conversion.
• Do not use loops.

Tasks

(a) Create the character vector and display its class.
(b) Convert it to numeric and display the new class.
(c) Calculate the total and the discounted total.
(d) Create c(1, "a", TRUE) and state its class, explaining why R chose it.

Answer:

```
prices <- c("120", "85.5", "200")
class(prices)
prices <- as.numeric(prices)
class(prices)
total <- sum(prices)
discounted <- total * 0.9
total
discounted
x <- c(1, "a", TRUE)
x
class(x)
```

Output:

```
[1] "character"
[1] "numeric"
[1] 405.5
[1] 364.95
[1] "1" "a" "TRUE"
[1] "character"

The class of c(1, "a", TRUE) is character. A vector can store only one
type of data, so R coerces all elements to the highest type present
(logical < numeric < character). Because "a" is a character, 1 becomes
"1" and TRUE becomes "TRUE".
```

Code of Conduct

I, K Sujith Sai Vighnesh / 192311199, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sujith Sai Vighnesh

Assessment Rubric

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs and constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient